



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

CSE 3175: ARTIFICIAL NEURAL NETWORKS

[Module 6]

Module -6	
RECURRENT HOPFIELD NETWORKS: Introduction, Operating Principles of the Hopfield Network, Stability Conditions of the Hopfield Network, Associative Memories, Outer Product Method, Pseudoinverse Matrix Method, Storage Capacity of Memories, Design Aspects of the Hopfield Network. Text Book 2: 7.1-7.5	4 Hours

33	Introduction to Operating Principles of the Hopfield Network	CO4
34	Associative Memories, Outer Product Method	CO4
35	Pseudoinverse Matrix Method, Storage Capacity of Memories	CO4
36	Design Aspects of the Hopfield Network	CO4



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Lecture-33

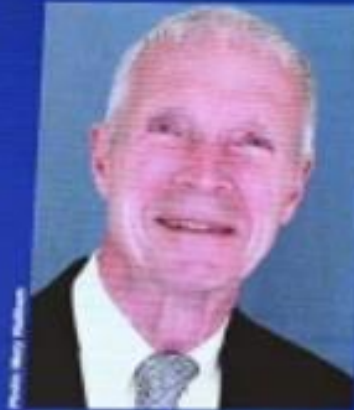
Introduction to Operating Principles of the Hopfield Network



NOBELPRISET I FYSIK 2024
THE NOBEL PRIZE IN PHYSICS 2024



KUNGL.
VETENSKAPS-
AKADEMIEN
THE ROYAL SWEDISH ACADEMY OF SCIENCES



John J. Hopfield

Princeton University, NJ, USA



Geoffrey E. Hinton

University of Toronto, Canada

"för grundläggande upptäckter och uppfinningar som möjliggör maskininlärning med artificiella neuronnätverk"

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"

- Nobel prize in physics 2024
- Significance - news

Introduction to Hopfield networks

- Proposed by John Hopfield in 1982
- Consists of one layer of 'n' fully connected recurrent neurons.
- All the neurons act as input and output
- All network neurons are connected to all others and to itself (all network outputs are fed back to all network inputs).
- Used in performing auto-association, optimization tasks and for different pattern recognition problems.
- The purpose of a Hopfield net is to store 1 or more patterns and to recall the full patterns based on partial input.

Hopfield Networks

- Artificial neural network to collect and retrieve memory like the human brain.
- Here, a neuron is either on or off the situation.
- The state of a neuron(on +1 or off 0) will be restored, relying on the input it receives from the other neuron.
- A Hopfield network is at first prepared to store various patterns or memories.
- Afterward, it is ready to recognize any of the learned patterns by uncovering partial or even some corrupted data about that pattern, i.e., it eventually settles down and restores the closest pattern.
- Thus, similar to the human brain, the Hopfield model has stability in pattern recognition.

Significance of Hopfield network

- Because they are unsupervised
- They do not learn using backpropagation. Instead they learn using biologically inspired, and very simple Hebbian Learning.
- They memorize patterns from just one sample. No need for a training set as we see in traditional machine learning. Very much what the brain does. Learning from one shot experiences
- They work as an auto associator and an error correction. Given a corrupted pattern or a few “parts” of a pattern, the hopfield net will be able to reconstruct it. Much like an autoencoder, with the difference that in the case of an autoencoder we do use backpropagation and a training set with hundreds of samples

- Consider the pattern: scan the input text and display corresponding ASCII



Original "T"

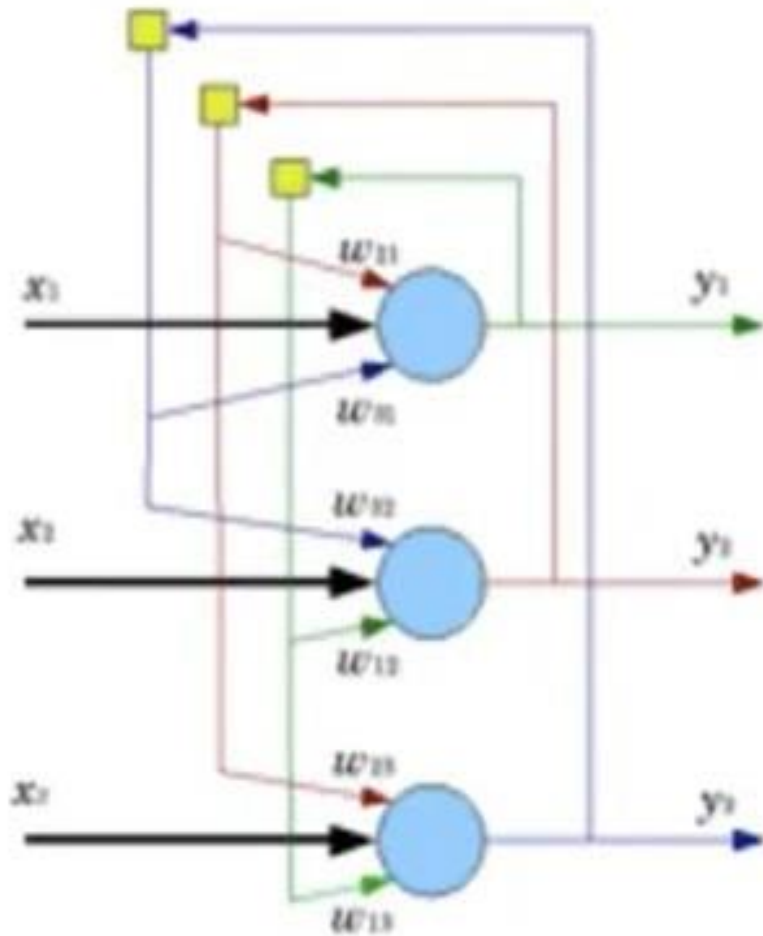


half of image
corrupted by
noise

Here's the way a Hopfield network would work.

- You map it out so that each pixel is one node in the network.
- You train it (or just assign the weights) to recognize each of the 26 characters of the alphabet, in both upper and lower case (that's 52 patterns).
- Now if your scan gives you a pattern like something on the right of the above illustration, you input it to the Hopfield network, and it chugs away for a few iterations, and eventually reproduces the pattern on the left, a perfect "T"

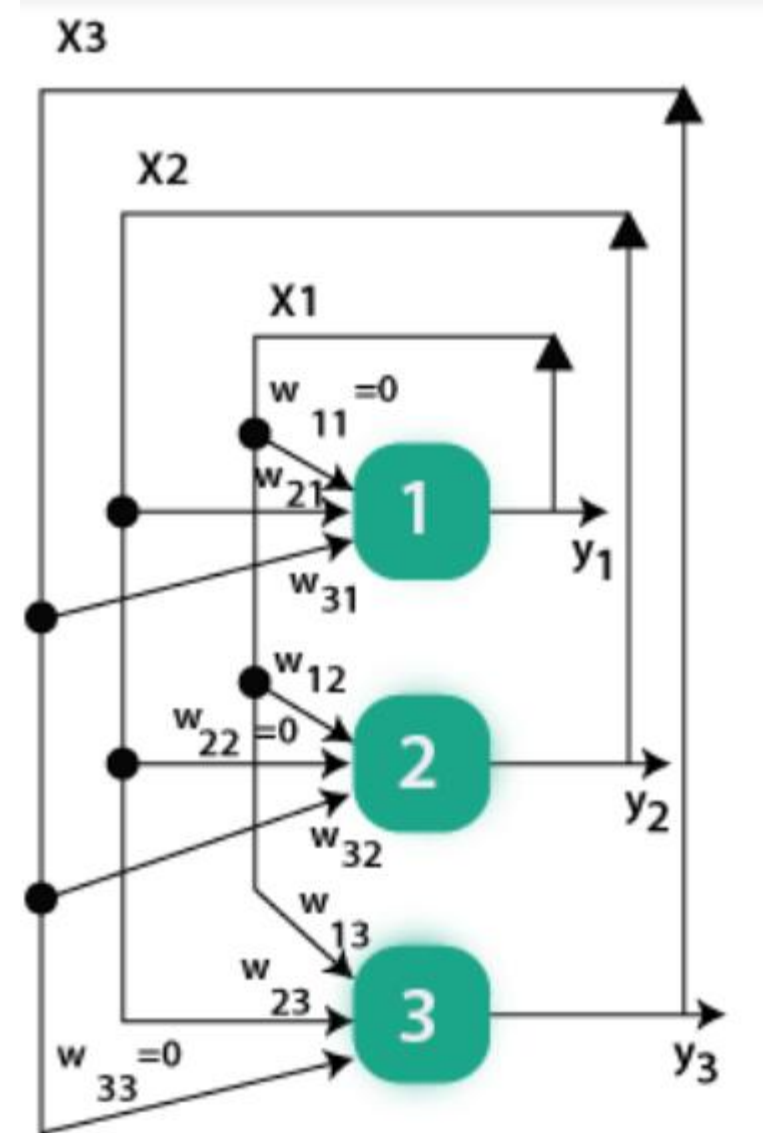
Architecture



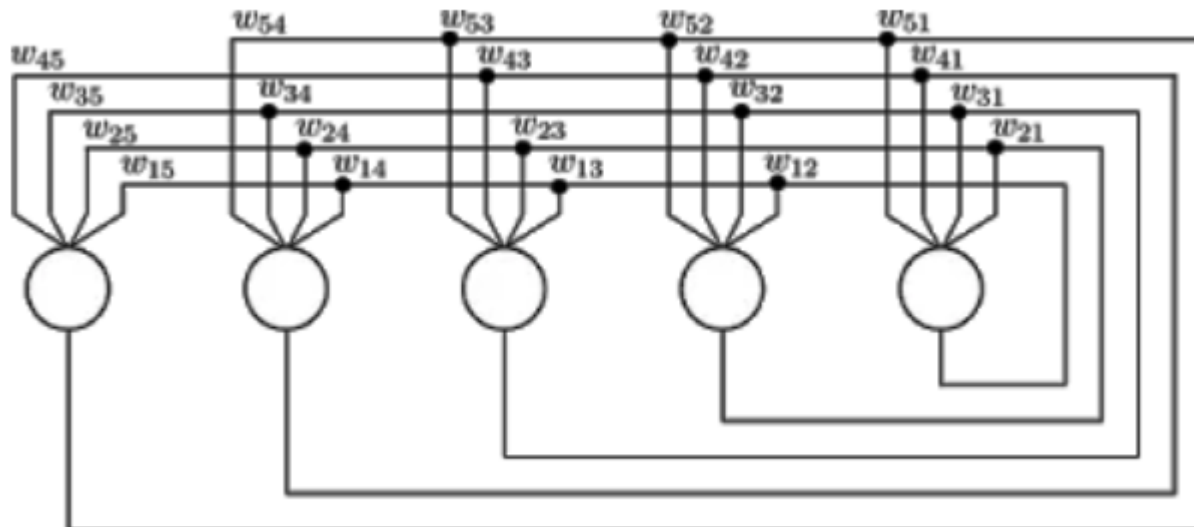
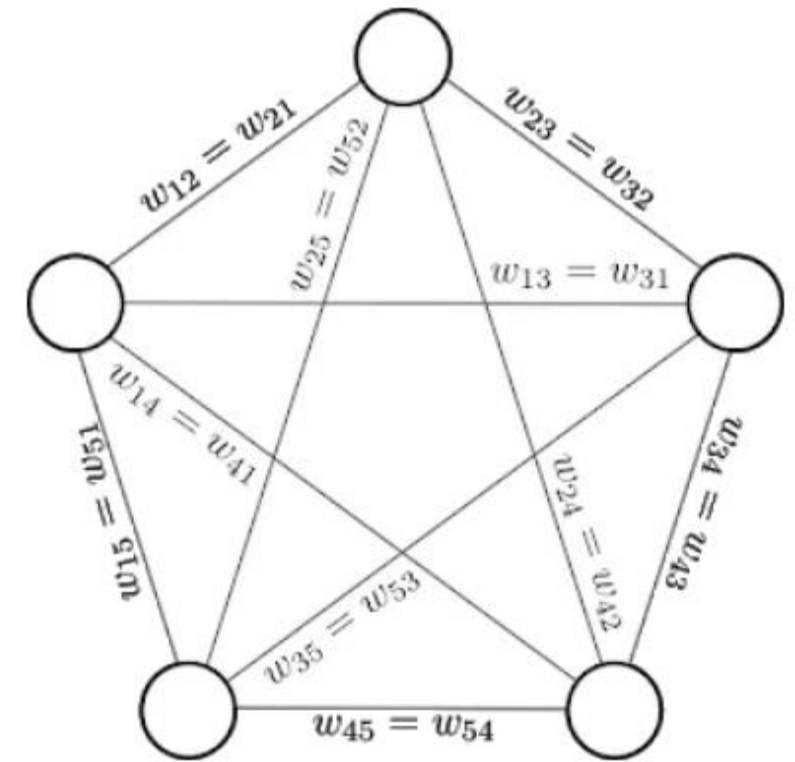
- All the nodes in a Hopfield network are both inputs and outputs, and they are fully interconnected.
- That is, each node is an input to every other node in the network.
- The Hopfield network(model) consists of a set of neurons and corresponding set of unit delays, forming a multiple loop feedback system as shown in Figure
- The number of feedback loops is equal to the number of neurons.
- Basically, the output of the neuron is feedback, via a unit delay element, to each of the other neurons in the network.
- No self feedback in the network.
- Weights should be symmetrical, i.e. $w_{ij} = w_{ji}$

Hopfield networks - Architecture

- A Hopfield net is a set of neurons that are:
- Bidirectionally connected between each other with symmetric weights, i.e. the weights between all neurons i and j are $w_{ij}=w_{ji}$.
- Not self-connected, this means that $w_{ii}=0$.
- With zero self-connectivity, $w_{ii}=0$ is given below.
Here, the given three neurons having values $i = 1, 2, 3$ with values $X_i=\pm 1$ have connectivity weight W_{ij} .



- A Hopfield network consisting of 5 neurons (right figure)
- This fails to capture it is the recurrence of the network. The Hopfield networks are recurrent because the inputs of each neuron are the outputs of the others, i.e. it possesses feedback loops
- A Hopfield network consisting of 5 neurons with feedback loops. (below figure)



Properties of Hopfield network

- A recurrent network with all nodes connected to all other nodes.
- Nodes have binary outputs (either 0,1 or -1,1).
- Weights between the nodes are symmetric.
- No connection from a node to itself is allowed.
- Nodes are updated asynchronously (i.e. nodes are selected at random).
- The network has no hidden nodes or layer.

How do they learn?

- Hopfield nets learn by using the very simple Hebbian rule.
- The Hebbian rule means that the value of a weight w_{ij} between two neurons, a_i and a_j is the product of the values of those neurons.
- Any two cells or systems of cells that are repeatedly active at the same time will tend to become ‘associated’ so that activity in one facilitates activity in the other.
- Neurons that fire together wire together, neurons out of sync, fail to link.

$$w_{ij} = a_i * a_j$$

$$w_{ij} = a_i * a_j$$

- So if a_i is say 10 and a_j is also 10, then the weight between them would also be 100.
- Also if both a_i and a_j had the value of -10, the weight would still be 100.
- Meaning that neurons that have the same value(or the same sign) would have a large positive weight.
- They “attract” each other.
- On the contrary, if say a_i was 10 and a_j was -10 then the weight between them would be -100.
- The neurons would repel each other in this case.

Feed forward in Hopfield networks

- Just like in any other neural net, we have learning, and feedforward.
- Once the network has learned something, we want to give that net a similar pattern and see the output.
- In the case of feedforward/LSTM nets used for classification, the output might be a discrete value.
- In the case of a hopfield net, the output is a reconstructed pattern.
- So in order to do feedforward in a hopfield net, we will initialize the neurons with the values of a pattern and then we will go through each neuron and compute the weighted sum of its inputs.
- If that sum has the same sign as the value of the neuron, we do nothing, otherwise we will flip the neuron.
- This is called to let the network “evolve” towards a learned pattern.

- **Start from the assumption that the values of each neuron in the net is either +1 or -1.**
- The weighted sum for any neuron a_i would look like this :

$$z = \sum_{\forall j \neq i} w_{ij} a_j$$

- Take the values of all neurons a_j that connect to a_i (in the case of a hopfield net that would be all the other neurons in the net since it is a complete graph) and we multiply each one by the weight w_{ij} between itself and a_i . We sum up all these values.
- In a hopfield net feed forwards procedure we are not interested in the actual value of that sum but in the sign on that sum.
- Then the value of that neuron will be :

$$a_i = \text{sign}(z)$$

- So lets say that if the value of the neuron is $+1$.
- If the weighted sum was positive then value of the neuron doesn't change.
- Otherwise if the sum was negative, then the value of the neuron would change from $+1$ to -1 .
- So in other words, the value of the neuron will change to match the sign of its weighted sum.
- We will do that for all the neurons in the net until we end un in a “stable” state.
- That is when the sign of any neuron will match the sign of it weighted sum.
- In other words, the networks will stop evolving.

Energy function in Hopfield network

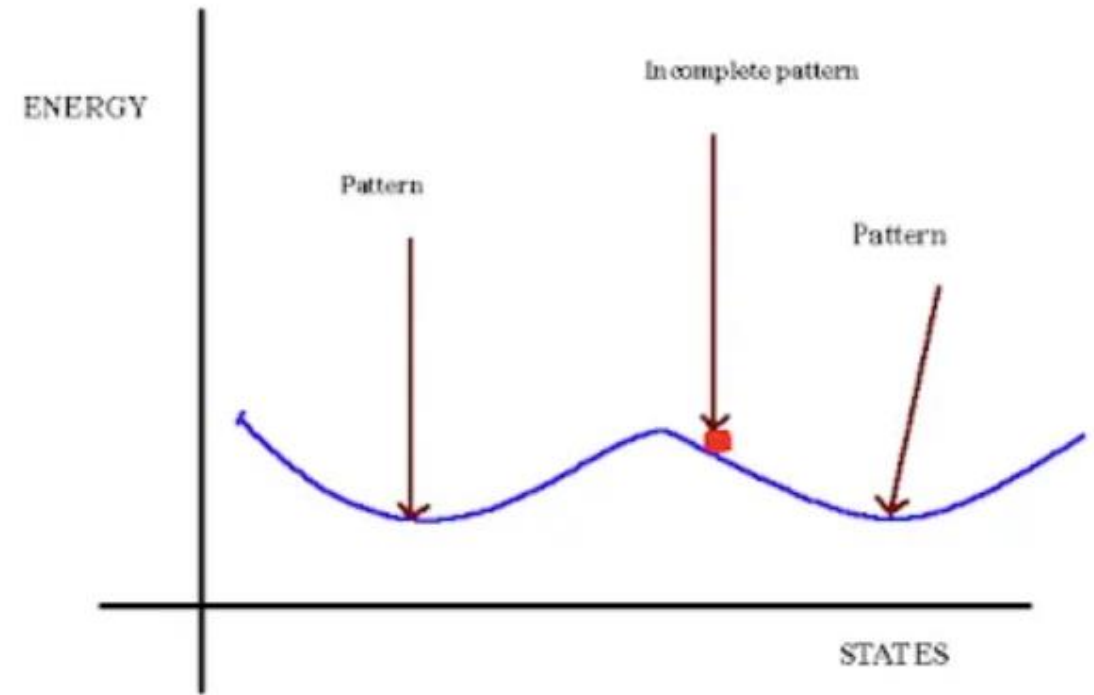
- Not a function of the weights of the network but rather a function of the states of the network (the values of the neurons)
- A cost function for a neural net measures the error between the actual output of the net given a sample and the desired output from the training set.
- And the idea is to minimize this function using gradient descent.

- In the case of a Hopfield network, we do not have any labeled training set. We just “look” at some patterns and memorize them.
- So we do not really have the notion of “error” since there is no “teacher” telling us the actual label of something.
- Think of the way we usually memorize experiences in our brain.
- We just experience them and this is how we learn them and then remember them.

$$E = -\frac{1}{2} \sum_i a_i \left(\sum_{j \neq i} w_{ij} a_j \right)$$

- The energy of a Hopfield net is the sum of each neuron times its weighted sum.
- We divide by 2 because the weights are symmetric.
- Every Flip of the neuron that we've seen in above will minimize the total energy of the net.
- So we can think of the surface of the energy function that is a function of the states of the network.
- If a net has N neurons, then the energy function will be a n dimensional function.

- Every learned pattern is a bowl in the surface of the energy function.
- And when we do feedforward starting with an incomplete pattern (the red dot in the image) we will flip the neurons until we will get to the valley, until the learned pattern is reconstructed.
- The reason why the learned patterns are the minimum of the energy function is because once we get to a stable state, where no neuron will flip.



Types of Hopfield networks

1. Discrete Hopfield Network

- A Hopfield network which operates in a discrete line fashion or in other words, it can be said the input and output patterns are discrete vector, which can be either binary 0,1 or bipolar +1,-1 in nature.
- The network has symmetrical weights with no self-connections i.e., $w_{ij} = w_{ji}$ and $w_{ii} = 0$.

2. Continuous Hopfield Network

- Continuous network has time as a continuous variable.
- It is also used in auto association and optimization problems such as travelling salesman problem.

- <https://medium.com/@serbanliviu/hopfield-nets-and-the-brain-e5880070cdba>



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Lecture-34

Training of Hopfield networks

Training of Discrete Hopfield network

During training of discrete Hopfield network, weights will be updated. As we know that we can have the binary input vectors as well as bipolar input vectors. Hence, in both the cases, weight updates can be done with the following relation

Case 1 – Binary input patterns

For a set of binary patterns **s_p , $p = 1$ to P**

Here, **$s_p = s_{1p}, s_{2p}, \dots, s_{ip}, \dots, s_{np}$**

Weight Matrix is given by

$$w_{ij} = \sum_{p=1}^P [2s_i(p) - 1][2s_j(p) - 1] \quad \text{for } i \neq j$$

Case 2 – Bipolar input patterns

For a set of bipolar patterns $\mathbf{s}_p, p = 1 \text{ to } P$

Here, $\mathbf{s}_p = \mathbf{s}_{1p}, \mathbf{s}_{2p}, \dots, \mathbf{s}_{ip}, \dots, \mathbf{s}_{np}$

Weight Matrix is given by

$$w_{ij} = \sum_{p=1}^P [s_i(p)][s_j(p)] \quad \text{for } i \neq j$$

Steps Involved in the training of a Hopfield Network are as mapped below:

- Initialize weights (w_{ij}) to store patterns (**using training algorithm**).
- For each input vector y_i , perform **steps 3-7**.
- Make the initial activators of the network equal to the external input vector x .

$$y_i = x_i : (for\ i = 1\ to\ n)$$

- For each vector y_i , perform **steps 5-7**.
- Calculate the total input of the network y_{in} using the equation given below.

$$y_{in_i} = x_i + \sum_j [y_j w_{ji}]$$

- Apply activation over the total input to calculate the output as per the equation given below:

$$y_i = \begin{cases} 1 & \text{if } y_{in} > \theta_i \\ y_i & \text{if } y_{in} = \theta_i \\ 0 & \text{if } y_{in} < \theta_i \end{cases}$$

(where θ_i (threshold) and is normally taken as 0)

- Now feedback the obtained output y_i to all other units. Thus, the activation vectors are updated.
- Test the network for convergence.

Example 1

Consider the following problem. We are required to create a Discrete Hopfield Network with the bipolar representation of the input vector as $[1 \ 1 \ 1 \ -1]$ or $[1 \ 1 \ 1 \ 0]$ (in case of binary representation) is stored in the network. Test the Hopfield network with missing entries in the first and second components of the stored vector (i.e. $[0 \ 0 \ 1 \ 0]$).

Given the input vector, $x = [1 \ 1 \ 1 \ -1]$ (bipolar) and we initialize the weight matrix ($\mathbf{w}_{ij}$) as:

$$\begin{aligned}
 w_{ij} &= \sum [s^T(p)t(p)] \\
 &= \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [1 \quad 1 \quad 1 \quad -1] \\
 &= \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix}
 \end{aligned}$$

and weight matrix with no self-connection is:

$$w_{ij} = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

As per the question, input vector x with missing entries, $x = [0 \ 0 \ 1 \ 0]$ (**[x1 x2 x3 x4]**) (binary). Make $y_i = x = [0 \ 0 \ 1 \ 0]$ (**[y1 y2 y3 y4]**). Choosing unit y_i (**order doesn't matter**) for updating its activation. Take the i^{th} column of the weight matrix for calculation.

(we will do the next steps for all values of y_i and check if there is convergence or not)

$$\begin{aligned} y_{in_1} &= x_1 + \sum_{j=1}^4 [y_j w_{j1}] \\ &= 0 + [0 \ 0 \ 1 \ 0] \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \end{bmatrix} \\ &= 0 + 1 \\ &= 1 \end{aligned}$$

$$= 1$$

Applying activation, $y_{in_1} > 0 \implies y_1 = 1$

giving feedback to other units, we get

$$y = [1 \quad 0 \quad 1 \quad 0]$$

which is not equal to input vector

$$x = [1 \quad 1 \quad 1 \quad 0]$$

Hence, no convergence. ‘

Now for next unit, we will take updated value via feedback. (i.e. $y = [1 \ 0 \ 1 \ 0]$)

$$\begin{aligned}y_{in_3} &= x_3 + \sum_{j=1}^4 [y_j w_{j3}] \\&= 1 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix} \\&= 1 + 1 \\&= 2\end{aligned}$$

Applying activation, $y_{in_3} > 0 \implies y_3 = 1$

giving feedback to other units, we get

$$y = [1 \ 0 \ 1 \ 0]$$

which is not equal to input vector

$$x = [1 \ 1 \ 1 \ 0]$$

Hence, no convergence.

Now for next unit, we will take updated value via feedback. (i.e. $y = [1 \ 0 \ 1 \ 0]$)

$$\begin{aligned}y_{in_4} &= x_4 + \sum_{j=1}^4 [y_j w_{j4}] \\&= 0 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} -1 \\ -1 \\ -1 \\ 0 \end{bmatrix} \\&= 0 + (-1) + (-1) \\&= -2\end{aligned}$$

Applying activation, $y_{in_4} < 0 \implies y_4 = 0$

giving feedback to other units, we get

$$y = [1 \ 0 \ 1 \ 0]$$

which is not equal to input vector

$$x = [1 \ 1 \ 1 \ 0]$$

Hence, no convergence.

Now for next unit, we will take updated value via feedback. (i.e. $y = [1 \ 0 \ 1 \ 0]$)

$$\begin{aligned}y_{in_2} &= x_2 + \sum_{j=1}^4 [y_j w_{j2}] \\&= 0 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} 1 \\ 0 \\ 1 \\ -1 \end{bmatrix} \\&= 0 + 1 + 1 \\&= 2\end{aligned}$$

Applying activation, $y_{in_2} > 0 \implies y_2 = 1$

giving feedback to other units, we get

$$y = [1 \ 1 \ 1 \ 0]$$

which is equal to input vector

$$x = [1 \ 1 \ 1 \ 0]$$

Hence, convergence with vector x .

Example 2

Numerical example

- For example, we have a 5 node Hopfield network and we want it to recognize the pattern (0 1 1 0 1).
- Since there are 5 nodes, we need a matrix of 5 x 5 weights, where the weights from a node back to itself are 0.
- The weight matrix will look like this: Symmetric matrix

0	W_{12}	W_{13}	W_{14}	W_{15}
W_{21}	0	W_{23}	W_{24}	W_{25}
W_{31}	W_{32}	0	W_{34}	W_{35}
W_{41}	W_{42}	W_{43}	0	W_{45}
W_{51}	W_{52}	W_{53}	W_{54}	0

Numerical example

- Calculate weight matrix:
- In this case, V is the vector $(0\ 1\ 1\ 0\ 1)$, so $V_1 = 0$, $V_2 = 1$, $V_3 = 1$, $V_4 = 0$, and $V_5 = 1$.
- Thus the computation of the weights is as follows:

$$W_{12} = (2V_1 - 1)(2V_2 - 1) = (0 - 1)(2 - 1) = (-1)(1) = -1$$

$$W_{13} = (2V_1 - 1)(2V_3 - 1) = (0 - 1)(2 - 1) = (-1)(1) = -1$$

$$W_{14} = (2V_1 - 1)(2V_4 - 1) = (0 - 1)(0 - 1) = (-1)(-1) = 1$$

$$W_{15} = (2V_1 - 1)(2V_5 - 1) = (0 - 1)(2 - 1) = (-1)(1) = -1$$

$$W_{23} = (2V_2 - 1)(2V_3 - 1) = (2 - 1)(2 - 1) = (1)(1) = 1$$

$$W_{24} = (2V_2 - 1)(2V_4 - 1) = (2 - 1)(0 - 1) = (1)(-1) = -1$$

$$W_{25} = (2V_2 - 1)(2V_5 - 1) = (2 - 1)(2 - 1) = (1)(1) = 1$$

$$W_{34} = (2V_3 - 1)(2V_4 - 1) = (2 - 1)(0 - 1) = (1)(-1) = -1$$

$$W_{35} = (2V_3 - 1)(2V_5 - 1) = (2 - 1)(2 - 1) = (1)(1) = 1$$

$$W_{45} = (2V_4 - 1)(2V_5 - 1) = (0 - 1)(2 - 1) = (-1)(1) = -1$$

Numerical example

- We get the full weight matrix

0	-1	-1	1	-1
-1	0	1	-1	1
-1	1	0	-1	1
1	-1	-1	0	-1
-1	1	1	-1	0

- This is for single pattern (0 1 1 0 1)

Example 3

Hopfield Network Algorithm

1. Assign connection weights

$$w_{ij} = \begin{cases} \sum_{s=0}^{M-1} x_i^s x_j^s & i \neq j \\ 0 & i = j, 0 \leq i, j \leq M-1 \end{cases}$$

where w_{ij} is the connection weight between node i and node j , and x_i^s is element i of the exemplar pattern for class s , and is either $+1$ or -1 . There are M patterns, from 0 to $M-1$, in total. The thresholds of the units are zero.

2. Initialise with unknown pattern

$$\mu_i(0) = x_i \quad 0 \leq i \leq N-1$$

where $\mu_i(t)$ is the output of node i at time t .

3. Iterate until convergence

$$\mu_i(t+1) = f_h \left[\sum_{j=0}^{N-1} w_{ij} \mu_j(t) \right] \quad 0 \leq j \leq N-1$$

The function f_h is the hard-limiting non-linearity, the step function, as in figure 3.3. Repeat the iteration until the outputs from the nodes remain unchanged.

Hopfield Network Algorithm

weight matrix: 5×5

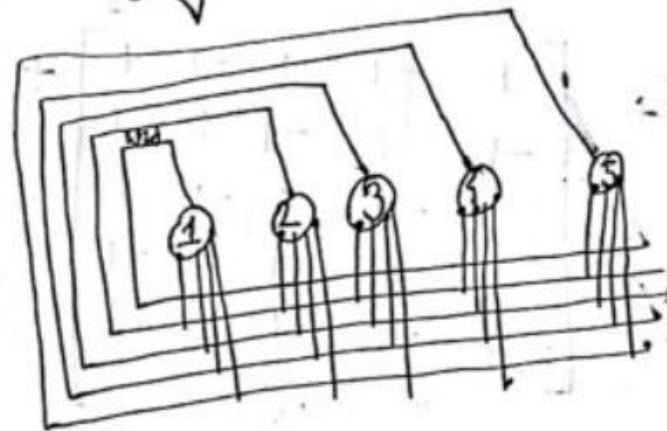
$$x^1 = \{-1, -1, 1, -1, 1\}$$

$$x^2 = \{1, -1, 1, -1, 1\}$$

$$x^3 = \{-1, 1, -1, 1, -1\}$$

$$x^4 = \{1, 1, 1, -1, -1\}$$

$$W = \sum_{s=0}^{M-1} (x^s)^T x^s$$



$$\begin{aligned}
 &= \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [-1 \ -1 \ 1 \ -1] + \begin{bmatrix} 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [1 \ 1 \ 1 \ -1 \ 1] \\
 &+ \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [-1 \ 1 \ -1 \ 1 \ -1] + \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} [1 \ 1 \ 1 \ -1 \ -1]
 \end{aligned}$$

$$= \begin{bmatrix} 1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 1 & -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \\ -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 \\ -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 \end{bmatrix} \\ + \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \\ -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 \\ -1 & 1 & -1 & 1 & -1 \\ 1 & -1 & 1 & -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 1 & -1 & 1 \\ -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & 1 & -1 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 0 & 2 & -2 & 0 \\ 0 & 4 & -2 & 2 & -4 \\ 2 & -2 & 4 & -4 & 2 \\ -2 & 2 & -4 & 4 & -2 \\ 0 & -4 & 2 & -2 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix}$$

Original pattern 0
Noisy pattern 0

$(-1, -1, 1, -1, 1)$
 $(-1, -1, -1, -1, 1)$

$$\mu(0) = [-1 \ -1 \ -1 \ -1 \ 1]_{1 \times 5}$$

$$W = \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -1 \\ 2 & -2 & 0 & -1 & 2 \\ -2 & 2 & -1 & 0 & -2 \\ 0 & -1 & 2 & -2 & 0 \end{bmatrix}$$

$$\text{Iteration-1: } \mu(0) = [-1 \ -1 \ -1 \ -1 \ 1] \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -1 \\ 2 & -2 & 0 & -1 & 2 \\ -2 & 2 & -1 & 0 & -2 \\ 0 & -1 & 2 & -2 & 0 \end{bmatrix}$$

$$= [0 \ -1 \ 6 \ 2 \ 1]$$

$$= [0 \ -1 \ 1 \ 1 \ 1]$$

$$\begin{aligned}
 \text{Iteration-2: } \mu(s) &= \begin{bmatrix} 0 & -1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -1 \\ 2 & -2 & 0 & -1 & 2 \\ -2 & 2 & -1 & 0 & -2 \\ 0 & -1 & 2 & -2 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 0 & -1 & 0 & -8 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 0 & -1 & 0 & -1 & 1 \end{bmatrix}
 \end{aligned}$$

$$\begin{aligned}
 \text{Iteration-3: } \mu(s) &= \begin{bmatrix} 0 & -1 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -1 \\ 2 & -2 & 0 & -1 & 2 \\ -2 & 2 & -1 & 0 & -2 \\ 0 & -1 & 2 & -2 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 2 & -6 & 8 & -1 & 6 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \end{bmatrix}
 \end{aligned}$$

Iteration-4:

$$\mu(0) = \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & 0 \\ 2 & -2 & 0 & -2 & 2 \\ -2 & 2 & 0 & 0 & -2 \\ 0 & -2 & 2 & -2 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & -8 & 10 & -10 & 8 \end{bmatrix}$$

Iteration-3 & Iteration-4 noisy pattern and previous pattern are matched. So recovered pattern $\begin{bmatrix} 1 & -1 & 1 & -1 & 1 \end{bmatrix}$

Original pattern	1	(1, -1, 1, -1, 1)
Noisy pattern	1	(1, -1, 1, -1, -1)

$$\mu(1) = \begin{bmatrix} 1 & -1 & 1 & -1 & -1 \end{bmatrix}_{1 \times 5}$$

$$\text{Iteration-1: } \mu(1) = \begin{bmatrix} 1 & -1 & 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 0 & 6 & -6 & 8 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 1 & -1 & 1 \end{bmatrix}$$

$$\text{Iteration-2: } \mu(1) = \begin{bmatrix} 1 & 0 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix}$$

Original pattern	1	(1, -1, 1, -1, 1)
Noisy pattern	1	(1, -1, 1, -1, -1)

$$\begin{aligned}
 &= [1 \ -8 \ 8 \ -8 \ 1] \\
 &= [1 \ -1 \ 1 \ -1 \ 1] \\
 \text{Iteration-3: } u(i) &= [1 \ -1 \ 1 \ -1 \ 1] \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix} \\
 &= [1 \ -8 \ 10 \ -10 \ 8] \\
 &= [1 \ -1 \ 1 \ -1 \ 1]
 \end{aligned}$$

Iteration-2 & Iteration-3 noisy pattern and previous pattern are equal. so recovered pattern $[1 \ -1 \ 1 \ -1 \ 1]$

Original pattern	2	(-1, 1, -1, 1, -1)
Noisy pattern	2	(-1, 1, -1, -1, -1)

$$\mu(2) = [-1 \ 1 \ -1 \ -1 \ -1]_{1 \times 5}$$

$$\text{Iteration-1: } \mu(2) = [-1 \ 1 \ -1 \ -1 \ -1] \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix}$$

$$= [0 \ 4 \ -2 \ 10 \ -4]$$

$$= [0 \ 1 \ -1 \ 1 \ -1]$$

$$\text{Iteration-2: } \mu(2) = [0 \ 1 \ -1 \ 1 \ -1] \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & -4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix}$$

$$= [-4 \ 8 \ -8 \ 8 \ -8]$$

$$= [-1 \ 1 \ -1 \ 1 \ -1]$$

Original pattern	2	$(-1, 1, -1, 1, -1)$
Noisy pattern	2	$(-1, 1, -1, -1, -1)$

$$\begin{aligned}
 \text{Iteration-3: } \mu(2) &= [-1 \ 1 \ -1 \ 1 \ -1] \begin{bmatrix} 0 & 0 & 2 & -2 & 0 \\ 0 & 0 & -2 & 2 & 4 \\ 2 & -2 & 0 & -4 & 2 \\ -2 & 2 & -4 & 0 & -2 \\ 0 & -4 & 2 & -2 & 0 \end{bmatrix} \\
 &= [-4 \ 8 \ -10 \ 10 \ -8] \\
 &= [-1 \ 1 \ -1 \ 1 \ -1]
 \end{aligned}$$

Iteration-2 & Iteration-3 noisy pattern and previous pattern are equal, so recovered pattern

$$[-1 \ 1 \ -1 \ 1 \ -1]$$

①

Hopfield Network

A Hopfield network stores information in the form of binary patterns by adjusting its connection weights. Here's how the information storage process works:

Define the Patterns: First, you need to define the binary patterns that you want the Hopfield network to store. These patterns represent the information or data you want to remember. Each pattern consists of binary values, such as +1/-1 or 0/1.

Initialize the Weight Matrix: The network starts with a symmetric weight matrix that is initially set to all zeros. The weight matrix has dimensions equal to the number of neurons in the network, which is determined by the length of the binary patterns. The diagonal elements of the weight matrix are set to zero.

Adjust the Weights: To store a pattern, you adjust the connection weights based on that pattern. The Hebbian learning rule is often used, which modifies the weights based on the correlation between the neuron states in the pattern.

- **Weights:**

- To store patterns $s(p)$, $p=1,2,...P$

bipolar:

$$w_{ij} = \sum_p s_i(p)s_j(p) \quad i \neq j$$
$$w_{ii} = 0$$

same as Hebbian rule (with zero diagonal)

binary:

$$w_{ij} = \sum_p (2s_i(p)-1)(2s_j(p)-1) \quad i \neq j$$
$$w_{ii} = 0$$

converting $s(p)$ to bipolar when constructing W .

- **Recall**

- Use an input vector to recall a stored vector
Each time, randomly select a unit for update

Recall Procedure

1. Apply recall input vector $\mathbf{x}$ to the network: $y_i := x_i \quad i = 1, 2, \dots, n$
2. While convergence = fails do
 1. **Randomly** select a unit
 - 2.2. Compute $y_{in_i} = x_i + \sum_{j \neq i} y_j w_{ji}$
 - 2.3. Determine activation of Y_i
$$y_i = \begin{cases} 1 & \text{if } y_{in} > \theta_i \\ y_i & \text{if } y_{in} = \theta_i \\ 0 & \text{if } y_{in} < \theta_i \end{cases}$$
 - 2.4. Periodically test for convergence.

Stability conditions of Hopfield Network

- The stability conditions of a Hopfield network refer to the conditions under which the network can successfully store and retrieve patterns while maintaining stable attractor states. The key stability conditions are as follows:

1. **Symmetric Weights:** The synaptic weights in a Hopfield network must be symmetric, meaning that the weight between neuron i and neuron j is equal to the weight between neuron j and neuron i . Mathematically, if w_{ij} is the weight between neuron i and neuron j , then w_{ij} must equal w_{ji} .
2. **Zero Diagonal Weights:** The diagonal weights (self-connections) in the Hopfield network should be set to zero. This ensures that neurons do not reinforce their own state and prevents spurious patterns from being stable attractor states.
3. **Bipolar Binary Neurons:** Neurons in a Hopfield network are typically modeled as bipolar binary units with states of $+1$ and -1 . This choice of encoding allows the network to find stable states and retrieve patterns effectively.
4. **Capacity Limit:** The number of patterns that can be reliably stored and retrieved by a Hopfield network is limited. The capacity of a Hopfield network depends on the network size and the quality of the patterns, and it is often less than the total number of neurons in the network.

Stability conditions of Hopfield Network

- 5. Energy Decrease: During the retrieval process, the energy of the network should decrease or remain stable as the network evolves. The network should converge to a stable attractor state with minimal energy.
- 6. Pattern Orthogonality: The patterns to be stored should be as orthogonal (uncorrelated) as possible. Patterns that are too similar may lead to interference, making it difficult for the network to retrieve the correct pattern.
- 7. Noise Tolerance: Hopfield networks are somewhat robust to noisy or partial inputs, but excessive noise may hinder pattern retrieval. The network should be able to converge to the correct attractor state even when presented with noisy or partially corrupted patterns.



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Lecture-35

Associative Memories, Design Aspects of the Hopfield Network.

Associative Memories

- An associative memory network can store a set of patterns as memories.
- When the associative memory is being presented with a key pattern, it responds by producing one of the stored patterns, which closely resembles or relates to the key pattern.
- Thus, the recall is through association of the key pattern, with the help of information memorized.
- These types of memories are also called as content-addressable memories (CAM).

Associative Memories

- The CAM can also be viewed as associating data to address, i.e.; for every data in the memory there is a corresponding unique address.
- Also, it can be viewed as data correlator. Here input data is correlated with that of the stored data in the CAM.
- The stored patterns must be unique, i.e., different patterns in each location.
- If the same pattern exists in more than one location in the CAM, then, even though the correlation is correct, the address is noted to be ambiguous.
- Associative memory makes a parallel search within a stored data file. The concept behind this search is to output any one or all stored items which match the given search argument.

Associative Memories

- There two types of associative memories
- **Auto Associative Memory**
- An auto-associative memory recovers a previously stored pattern that most closely relates to the current pattern. It is also known as an auto-associative correlator.
- In the auto associative memory network, the training input vector and training output vector are the same

- **Hetero Associative memory**

- In a hetero-associate memory, the training input and the target output vectors are different.
- The weights are determined in a way that the network can store a set of pattern associations.
- The association here is a pair of training input target output vector pairs ($s(p)$, $t(p)$), with $p = 1, 2, \dots, p$.
- Each vector $s(p)$ has n components and each vector $t(p)$ has m components.
- The determination of weights is done either by using Hebb rule or delta rule.
- The net finds an appropriate output vector, which corresponds to an input vector x , that may be either one of the stored patterns or a new pattern.

Associative Memories

- In the context of **Hopfield Neural Networks**, **Associative Memories** enable the system to store and recall patterns.
- The Hopfield network is an example of a recurrent neural network designed specifically for associative memory tasks, where the goal is to store a set of patterns and retrieve them when presented with partial or noisy versions of those patterns.
- This is achieved using Hebbian learning and the Outer Product Method.

Associative Memory in Hopfield Networks

In a Hopfield network, the associative memory works by creating a dynamical system where each stored pattern is an attractor. When an input pattern is presented, the network evolves (through iterative updates) toward one of the stored patterns (the attractor) that closely matches the input.

Outer Product Method in Hopfield Networks

The **Outer Product Method** is used to define the weight matrix in Hopfield networks, which stores the desired patterns for retrieval. The Hopfield network operates by iterating over the state of neurons to minimize an energy function, which leads the system toward a stable state (a stored pattern).

Outer Product Rule

Outer products rule is a method for finding weights of an associative net.

Input=> $s = (s_1, \dots, s_i, \dots, s_n)$

Output=> $t = (t_1, \dots, t_j, \dots, t_m)$

The outer product of the two vectors is the product of the matrices $S = s^T$ and $T = t$, i.e., between $[n \times 1]$ matrix and $[1 \times m]$ matrix. The transpose is to be taken for the input matrix given.

$$ST = s^T t \Rightarrow \begin{bmatrix} s_1 \\ \vdots \\ s_i \\ \vdots \\ s_n \end{bmatrix} * \begin{bmatrix} t_1 & \dots & t_j & \dots & t_m \end{bmatrix}$$

This weight matrix is same as the weight matrix obtained by Hebb rule to store the pattern association s:t. For storing a set of associations, $s(p):t(p)$, $p = 1$ to P , wherein,

$s(p) = (s_1(p), \dots, s_i(p), \dots, s_n(p))$

$t(p) = (t_1(p), \dots, t_j(p), \dots, t_m(p))$

the weight matrix $W = \{w_{ij}\}$ can be given as

$$w = \sum_{p=1}^n s^T(p) \cdot s(p)$$

Steps of the Outer Product Method in Hopfield Networks:

1. Pattern Representation:

Each pattern to be stored is represented as a binary vector. For simplicity, the binary values are often taken as $+1$ and -1 rather than 1 and 0 . Assume we have p patterns, each of length n , represented as:

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p \quad \text{where} \quad \mathbf{x}_i \in \{+1, -1\}^n$$

2. Weight Matrix Calculation (Outer Product Rule):

The weight matrix W is constructed by the sum of the outer products of each pattern with itself:

$$W = \sum_{i=1}^p \mathbf{x}_i \mathbf{x}_i^T$$

Here, $\mathbf{x}_i^T$ is the transpose of the vector $\mathbf{x}_i$, and the outer product $\mathbf{x}_i \mathbf{x}_i^T$ generates a matrix where each element represents the interaction between pairs of neurons for that specific pattern.

- The element W_{jk} of the weight matrix represents the connection strength between neuron j and neuron k .
- If $W_{jk} > 0$, the neurons have a positive relationship (excitatory).
- If $W_{jk} < 0$, the neurons have a negative relationship (inhibitory).

3. Zeroing the Diagonal:

To prevent self-feedback in the network, the diagonal elements of the weight matrix (which represent self-connections) are set to zero:

$$W_{ii} = 0 \quad \forall i$$

This ensures that a neuron does not directly affect itself.

4. Energy Function:

The Hopfield network operates by minimizing an energy function E , which guides the system toward a stable state (a stored pattern). The energy function is given by:

$$E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n W_{ij} s_i s_j$$

where s_i is the state of neuron i , and W_{ij} is the weight between neurons i and j . The network updates the states of the neurons to minimize this energy function, leading it to converge to one of the stored patterns.

5. Pattern Recall:

When an input pattern $\mathbf{s}$ (which can be noisy or incomplete) is presented to the network, the recall process involves updating the state of each neuron according to the following rule:

$$s_j^{\text{new}} = \text{sign} \left(\sum_{k=1}^n W_{jk} s_k \right)$$

The neurons update their states iteratively until the network converges to a stable state, which is ideally one of the stored patterns. This is a form of content-addressable memory: the network can recall the entire pattern based on a partial or corrupted input.

Key Characteristics of the Outer Product Method in Hopfield Networks:

- **Capacity Limitations:**

The outer product method works well when the number of stored patterns is much smaller than the number of neurons. If too many patterns are stored, spurious states (non-stored patterns) may emerge during recall.

- **Error Correction:**

The Hopfield network can correct small errors in the input pattern and still recall the correct stored pattern. This is due to the attractor dynamics of the network, where small perturbations in the input lead the network back to the stored pattern.

Applications of Hopfield Networks with Outer Product Method:

- Pattern Completion:**

The network can recover full patterns from noisy or partial inputs, which is useful in image recognition, signal processing, and other fields.

- Memory Storage:**

It can be used to store binary data (like black-and-white images) and recall them when provided with an incomplete or corrupted version of the data.

- Error Correction:**

The Hopfield network's ability to converge to stored patterns makes it useful in applications requiring error detection and correction, such as in communication systems.

Design aspects of Hopfield network

The design aspects of a Hopfield Network involve a variety of considerations ranging from its network architecture to its learning and recall mechanisms. Below are the critical design elements that define how a Hopfield network operates and the factors influencing its efficiency, performance and application.

1. Architecture

The architecture of a Hopfield network is simple yet foundational to its design. Some key features include:

- **Fully Connected Neurons:** Every neuron in a Hopfield network is connected to every other neuron. This full connectivity is crucial for the associative memory behavior, where any neuron can influence any other neuron. The connections are symmetric, meaning the weight from neuron i to neuron j is the same as the weight from neuron j to neuron i .

$$W_{ij} = W_{ji}, \quad W_{ii} = 0$$

The diagonal elements of the weight matrix are zero, as a neuron is not connected to itself.

- **Binary Neurons:** Hopfield networks typically use binary neurons, where each neuron takes one of two possible states: $+1$ or -1 (sometimes 1 and 0). Neurons update based on the weighted sum of their inputs and their activation threshold.
- **Energy Function:** The network minimizes a defined energy function, which governs how the neurons settle into stable states. The energy function E is given by:

$$E = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N W_{ij} x_i x_j$$

This energy function ensures the network moves toward lower energy states, which correspond to the stored patterns (memories).

2. Learning Rule (Weight Update)

The weights in the Hopfield network are typically determined using one of two main methods:

a. **Hebbian Learning (Outer Product Rule):**

- This is the most basic learning rule used in Hopfield networks, based on Hebb's rule: "Neurons that fire together, wire together."
- For each pair of neurons i and j , the weight W_{ij} is calculated using the outer product of the patterns to be stored.

$$W_{ij} = \frac{1}{N} \sum_{k=1}^p x_i^{(k)} x_j^{(k)}$$

Where:

- p is the number of patterns to be stored.
- $x_i^{(k)}$ is the state of neuron i in the k -th pattern.

The simplicity of the outer product rule makes it computationally efficient, but it limits the network's capacity to store patterns.

b. Pseudoinverse Method:

- As discussed earlier, this method is more computationally complex but improves the network's capacity to store patterns by solving for the exact weight matrix that minimizes recall errors.
- The pseudoinverse is used to adjust the weights to more accurately store and recall patterns, especially when there are many or highly correlated patterns.

3. Activation Function

The neurons in a Hopfield network use a **discrete** activation function. The state of neuron i is updated based on the total input it receives from other neurons. The neuron's state is updated asynchronously (one at a time) or synchronously (all neurons update at once), depending on the design:

$$x_i = \text{sign} \left(\sum_j W_{ij} x_j \right)$$

- The sign function returns $+1$ if the weighted sum is positive, and -1 otherwise.
- This discrete update mechanism drives the network toward a stable state that represents one of the stored patterns (or a spurious state).

4. Energy Minimization and Stability

The energy minimization property of the Hopfield network is a core design aspect. The network converges to a stable state by minimizing the energy function, which ensures that the network "settles" into one of its stored memories:

- Each update reduces the overall energy of the system.
- The network can converge to **local minima**, which represent stored patterns.
- However, **spurious states** may also exist, which are stable states that do not correspond to any stored memory. These arise from the interaction between multiple stored patterns.

The network's ability to converge to correct stable states depends on the proper design of the weight matrix and how many patterns are stored relative to the number of neurons.

5. Memory Storage and Recall

- **Auto-associative Memory:** Hopfield networks are auto-associative, meaning they can retrieve an entire pattern based on a partial or noisy input. This feature makes the network useful for pattern recognition tasks.
- **Pattern Recall:** Given a noisy or incomplete input, the network iterates, updating neuron states, and eventually converges to one of the stored patterns, provided that the input is not too different from the stored pattern. This is known as **content-addressable memory**.

6. Capacity and Scalability

The storage capacity of the Hopfield network is limited by its architecture and learning rule:

- **Outer Product Rule:** The theoretical limit for storage capacity is approximately $0.15N$, where N is the number of neurons in the network. Storing more patterns than this results in a high likelihood of spurious states and retrieval errors.
- **Pseudoinverse Method:** This method increases the storage capacity to N patterns but at the cost of greater computational complexity.

7. Noise Tolerance and Robustness

Hopfield networks are moderately robust to noise. If an input pattern is close enough to one of the stored patterns, the network will typically converge to the correct memory. However, if the input is too noisy or distorted, the network may converge to an incorrect pattern or a spurious state. The design trade-offs to improve noise tolerance include:

- **Increasing the number of neurons:** A larger network is generally more tolerant of noise, but this comes with increased computational demands.
- **Sparsity in pattern encoding:** Sparsely encoded patterns can help reduce interference between patterns, increasing noise tolerance and storage capacity.

8. Convergence Dynamics

The convergence behavior of a Hopfield network depends on the updating mechanism used for neuron states:

- **Asynchronous Updates:** In this approach, one neuron is updated at a time, chosen randomly or in sequence. This method guarantees convergence to a stable state (a local energy minimum), but can be slower in large networks.
- **Synchronous Updates:** All neurons are updated simultaneously based on the current states of all other neurons. While this method is faster, it may not always guarantee convergence, and oscillations or other unstable behavior can occur in some cases.

9. Spurious States and Attractors

Spurious states are local minima in the energy function that do not correspond to any of the stored patterns. These states can arise due to:

- **Interference between patterns:** When too many patterns are stored, or if they are highly correlated, the network can create spurious attractors that represent mixed or false memories.
- **Storage capacity limits:** Storing more than the network's capacity results in the emergence of these spurious states, reducing the reliability of pattern recall.

10. Applications

The design aspects of Hopfield networks make them useful for a variety of applications:

- **Pattern recognition and classification:** Hopfield networks can classify patterns by converging on stored memories that represent classes.
- **Optimization problems:** The energy minimization property makes them suitable for solving optimization problems like the **Traveling Salesman Problem (TSP)**, where the goal is to find a solution that minimizes a cost function.
- **Associative memory:** Hopfield networks are used in situations requiring associative recall, such as recognizing images, text, or speech from incomplete inputs.

Summary of Design Aspects:

1. **Architecture:** Fully connected, symmetric neurons with binary states.
2. **Learning Rule:** Hebbian learning (outer product) or pseudoinverse method for weight calculation.
3. **Activation Function:** Discrete update based on input from other neurons.
4. **Energy Minimization:** Governs convergence to stable states (stored memories).
5. **Memory Storage:** Limited to $0.15N$ patterns with the outer product rule, but can be increased to N with the pseudoinverse method.
6. **Noise Tolerance:** Moderately robust, but depends on the network's design.
7. **Convergence:** Asynchronous updates guarantee convergence, while synchronous updates may cause oscillations.
8. **Spurious States:** Undesirable stable states can emerge when storing too many patterns.



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Lecture-36

Pseudoinverse Matrix Method Storage Capacity of Memories

Pseudoinverse Matrix Method

- The Pseudoinverse Matrix Method is an advanced technique used in Hopfield Networks to improve memory storage capacity and recall accuracy beyond the simpler Outer Product Method.
- While the outer product method works well for small numbers of patterns, its effectiveness diminishes as the number of stored patterns increases.
- The pseudoinverse method is used to better handle the overlap of stored patterns, enabling more reliable pattern recall when the network is closer to its storage limit.

Overview of the Pseudoinverse Matrix Method

In a Hopfield Network, the goal is to store a set of binary patterns $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(p)}$ such that the network can recall the correct pattern when given a noisy or incomplete version. The weight matrix W needs to be calculated in a way that minimizes interference between patterns.

The **pseudoinverse method** attempts to address this by directly solving for a weight matrix that will recall the stored patterns exactly, even when there are many overlapping patterns.

Step-by-Step Process

1. **Define the Patterns:** You have p binary patterns $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(p)}$, each with N elements. Each pattern can be represented as a column vector, and all patterns together form a matrix X , where:

$$X = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(p)}]$$

Here, X is an $N \times p$ matrix (where N is the number of neurons and p is the number of patterns).

2. **Goal:** The goal is to find a weight matrix W such that the network can recall any stored pattern $\mathbf{x}^{(i)}$ when it is given as an input. Mathematically, this means:

$$\mathbf{x}^{(i)} = W\mathbf{x}^{(i)}$$

However, for multiple patterns, interference between stored patterns can cause issues. The pseudoinverse method minimizes this interference.

3. **Calculate the Pseudoinverse of the Pattern Matrix:** The pseudoinverse X^+ of the pattern matrix X is used to directly solve for the weight matrix. The pseudoinverse can be computed using the formula:

$$X^+ = (X^T X)^{-1} X^T$$

Where:

- X^T is the transpose of X
- $(X^T X)^{-1}$ is the inverse of the matrix $X^T X$, provided it is invertible.

X^+ is an $N \times p$ matrix that represents the best linear approximation to the inverse of X .

4. **Compute the Weight Matrix:** Once you have the pseudoinverse X^+ , the weight matrix W can be calculated as:

$$W = X X^+$$

This weight matrix minimizes the error when recalling the stored patterns, effectively improving the network's capacity and accuracy in pattern recall.

- **Symmetry:** To ensure the weight matrix is symmetric (a requirement for Hopfield networks), we take the symmetric part of W :

$$W = \frac{1}{2}(W + W^T)$$

5. **Zero Out the Diagonal:** As with the outer product method, the diagonal elements of the weight matrix must be zero (since a neuron shouldn't have a connection to itself):

$$W_{ii} = 0 \quad \forall i$$

Why Use the Pseudoinverse Method?

- **Higher Capacity:** The pseudoinverse method allows the Hopfield network to store more patterns than the outer product method. While the outer product method can store approximately $0.15N$ patterns (where N is the number of neurons), the pseudoinverse method can store up to N patterns without significant degradation in recall accuracy.
- **Exact Recall:** The pseudoinverse method ensures that the stored patterns are exact solutions to the network's dynamics. That means the network is guaranteed to recall the correct pattern if the input is not too far from the stored pattern.

- **Handles Overlap:** In real-world applications, many patterns will have overlapping or correlated elements. The pseudoinverse method is designed to handle such overlap more effectively than the outer product method, minimizing interference between patterns.

Example Calculation

Let's assume we want to store two patterns in a Hopfield network with 4 neurons:

- $\mathbf{x}^{(1)} = [1, -1, 1, -1]$
- $\mathbf{x}^{(2)} = [-1, 1, -1, 1]$

1. Pattern Matrix X :

$$X = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

2. Pseudoinverse of X :

$$X^T X = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}$$

$$(X^T X)^{-1} = \begin{bmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{bmatrix}$$

Then,

$$X^+ = (X^T X)^{-1} X^T = \begin{bmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{bmatrix}$$

3. Weight Matrix W :

$$W = X X^+ = \begin{bmatrix} 0 & -2 & 2 & -2 \\ -2 & 0 & -2 & 2 \\ 2 & -2 & 0 & -2 \\ -2 & 2 & -2 & 0 \end{bmatrix}$$

This weight matrix is more robust than the one generated by the outer product method, especially when there are more patterns.

To analyze how the Pseudoinverse method is used to create the weight matrix for a Hopfield network storing the patterns $x^{(1)} = [1, -1, 1, -1]$ and $x^{(2)} = [-1, 1, -1, 1]$, we follow these steps mathematically.

Step 1: Constructing the Pattern Matrix

We define the pattern matrix X using the given patterns:

$$X = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

Step 2: Compute X^T (Transpose of X)

We compute the transpose of X :

$$X^T = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Step 3: Calculate $X^T \cdot X$

Next, we calculate the product $X^T \cdot X$:

$$X^T \cdot X = \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}$$

Step 4: Compute the Inverse of $X^T \cdot X$

We need to compute the inverse of $X^T \cdot X$:

$$(X^T \cdot X)^{-1} = \frac{1}{\det(A)} \cdot \text{adj}(A)$$

Where:

- $A = X^T \cdot X = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}$
- $\det(A) = 4 \cdot 4 - (-4) \cdot (-4) = 16 - 16 = 0$

Since the determinant is zero, $X^T \cdot X$ is not invertible. In practice, for Hopfield networks, we typically use a modified approach to compute the pseudoinverse when $X^T \cdot X$ is singular. We can calculate the pseudoinverse directly.

Step 5: Compute the Pseudoinverse X^+

Using the formula for the pseudoinverse:

$$X^+ = (X^T \cdot X)^{-1} \cdot X^T$$

Instead, we compute the pseudoinverse in a manner suited for this context:

$$X^+ = X^T \cdot (XX^T)^{-1}$$

Compute XX^T

$$XX^T = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}$$

As noted, the matrix XX^T is also singular. To proceed, we can compute the pseudoinverse by recognizing that for Hopfield networks, the weight matrix is defined as:

Step 6: Compute the Weight Matrix W

For Hopfield networks, the weight matrix can be directly computed as follows:

$$W = \sum_{p=1}^2 x^{(p)} \cdot (x^{(p)})^T$$

Calculating each term:

1. For $x^{(1)} = [1, -1, 1, -1]$:

$$x^{(1)} \cdot (x^{(1)})^T = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

2. For $x^{(2)} = [-1, 1, -1, 1]$:

$$x^{(2)} \cdot (x^{(2)})^T = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

Final Weight Matrix Calculation

Adding these two matrices together gives:

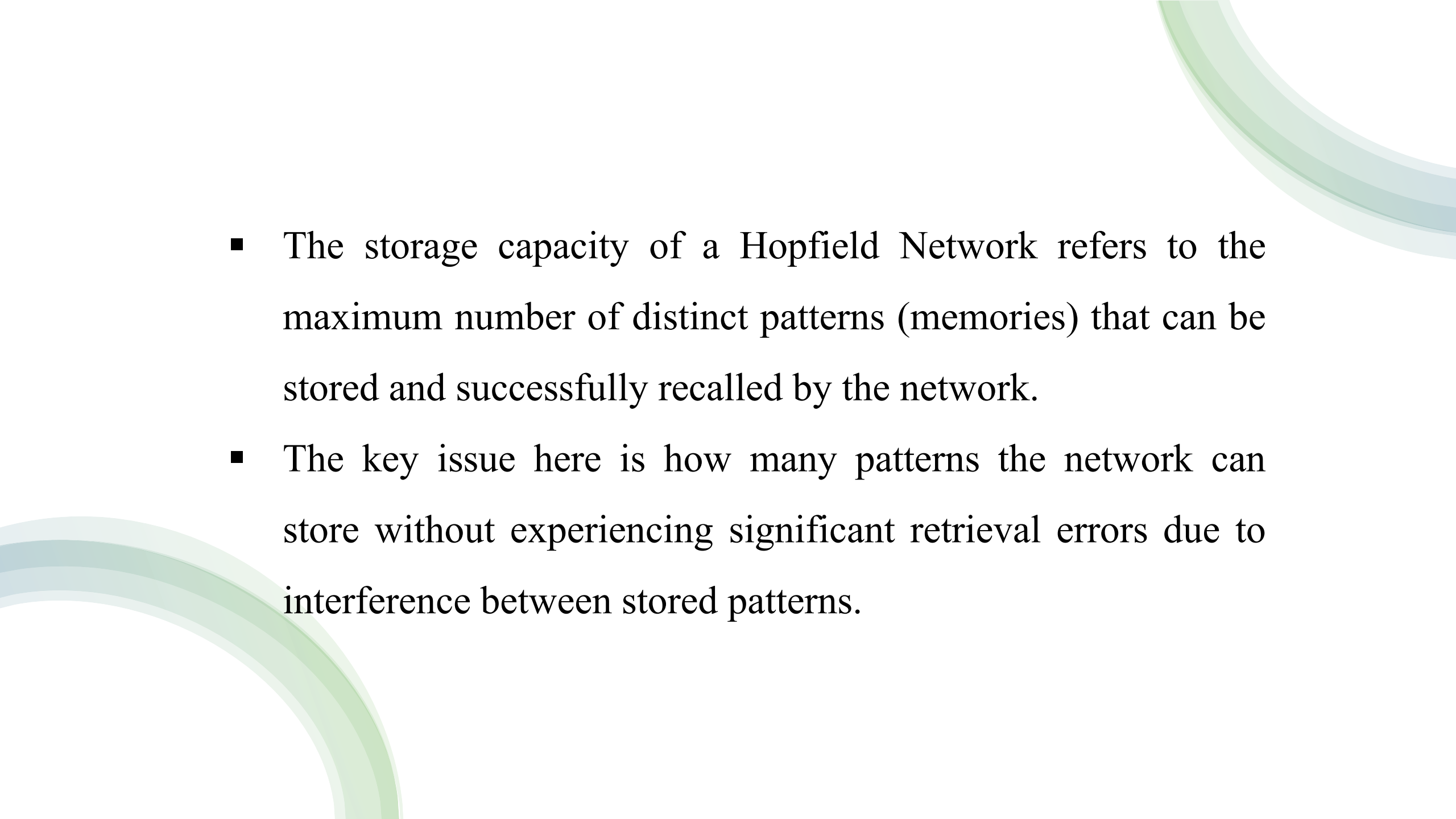
$$W = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Comparison with Outer Product Method

- **Outer Product Method:** Simple but less accurate when storing many patterns. Capacity is limited to about $0.15N$.
- **Pseudoinverse Method:** More complex but provides exact recall of stored patterns and can store up to NNN patterns. It deals better with overlapping patterns and offers higher storage capacity.

Limitations of the Pseudoinverse Method

- **Computation Complexity:** Calculating the pseudoinverse, especially for large networks, can be computationally expensive compared to the outer product method.
- **Sensitivity to Noise:** While the pseudoinverse method ensures exact recall, it may be more sensitive to noise in the input compared to the outer product method.

- 
- The storage capacity of a Hopfield Network refers to the maximum number of distinct patterns (memories) that can be stored and successfully recalled by the network.
 - The key issue here is how many patterns the network can store without experiencing significant retrieval errors due to interference between stored patterns.

Key Concepts Influencing Storage Capacity

1. Neurons and Memory Patterns:

- Let N be the number of neurons in the network.
- The network stores binary patterns $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(p)}$, each of size N (where p is the number of patterns).

2. Sparsity of Patterns:

- The patterns are often binary, represented by +1 or -1 (or equivalently, 1 and 0). Patterns can be dense (equal probabilities for +1 and -1) or sparse (a low fraction of neurons are active, i.e., have a value of +1).

3. Overlap and Interference:

- If the stored patterns are too similar (high overlap), interference between them may cause the network to converge to incorrect patterns during recall, reducing effective storage capacity.

Theoretical Storage Capacity (for Fully Connected Hopfield Networks)

1. Hebbian Learning (Outer Product Method):

Using the **outer product method**, the storage capacity of a Hopfield network has a theoretical limit of about:

$$p_{\max} \approx 0.15N$$

This means that a network with N neurons can store about $0.15N$ patterns. For example:

- If the network has 100 neurons, it can store about 15 patterns.
- If the network has 1000 neurons, it can store about 150 patterns.

This capacity limit arises from **pattern interference**. When too many patterns are stored, spurious states (incorrect or mixed patterns) can emerge, making it difficult for the network to recall the correct memory.

2. Pseudoinverse Method:

The pseudoinverse method improves the network's ability to recall stored patterns, leading to a higher capacity. The storage capacity using the pseudoinverse method is closer to:

$$p_{\max} \approx N$$

This means a Hopfield network using the pseudoinverse method can store approximately N patterns (where N is the number of neurons). For example:

- A network with 100 neurons can store close to 100 patterns.

This is a significant improvement over the outer product method, but the pseudoinverse method requires more computational resources and is less commonly used due to its complexity.

3. Sparsely Coded Networks:

In cases where the patterns are **sparsely coded** (most of the neurons are inactive, meaning they have a value of -1 or 0), the storage capacity can be much higher. In such cases, the network can store more patterns because the reduced overlap between patterns reduces interference.

For sparse patterns, the capacity can reach:

$$p_{\max} \propto \frac{N^2}{2}$$

In such scenarios, the Hopfield network can theoretically store a number of patterns proportional to the square of the number of neurons, but this depends heavily on the degree of sparsity and the network's architecture.

Practical Considerations for Storage Capacity

1. **Noise Tolerance:** As the number of stored patterns approaches the maximum storage capacity, the network becomes more sensitive to noise and partial inputs. This means that for a network storing patterns close to its capacity limit, even small amounts of noise in the input can lead to incorrect recall.
2. **Spurious States:** If too many patterns are stored, the Hopfield network can develop **spurious states**, which are stable states that do not correspond to any of the original stored patterns. These spurious states are combinations or distortions of the original patterns.
3. **Energy Landscape:** The capacity is also influenced by the **energy landscape** of the network. The Hopfield network minimizes an energy function, and the stored patterns correspond to local minima of this function. As more patterns are stored, the energy landscape becomes more complex, leading to additional local minima (spurious states), which reduces the network's ability to correctly recall the original patterns.

Enhancing Storage Capacity

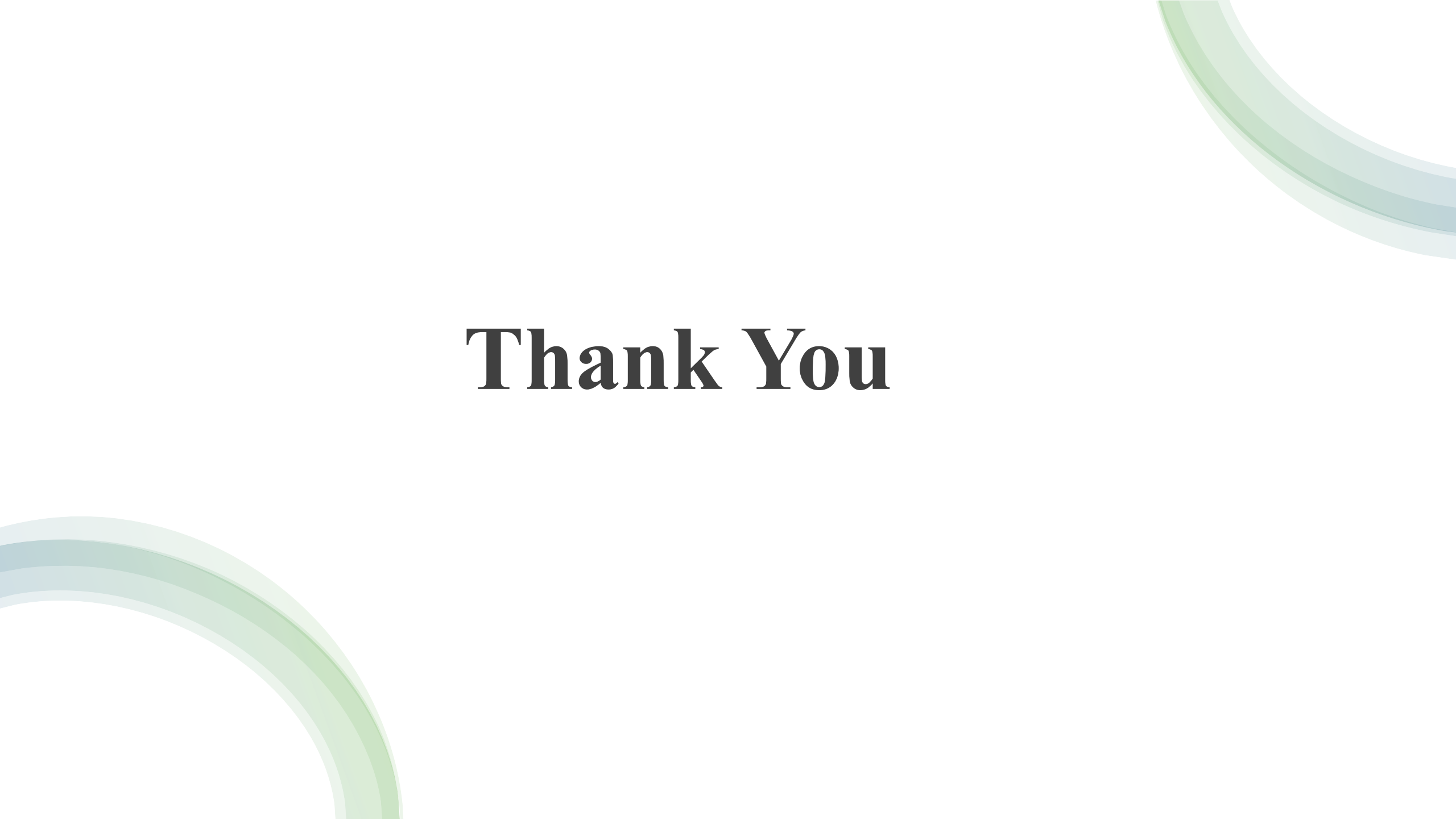
- To enhance the storage capacity of a Hopfield network, several strategies can be employed:

1.Pseudoinverse Method: As mentioned earlier, the pseudoinverse method improves capacity by directly solving for a weight matrix that can exactly recall each stored pattern.

2.Stochastic Networks: Networks that include some degree of randomness, like **Boltzmann machines**, can handle larger pattern capacities by allowing for a probabilistic recall mechanism. These networks don't directly minimize the Hopfield energy function but instead employ a stochastic process to find likely stored patterns.

3. Sparsely Coded Patterns: By using sparse patterns, where only a small subset of neurons are active, the storage capacity can be dramatically increased because sparse patterns have less overlap, reducing interference.

4. Using More Complex Network Architectures: More advanced network structures like **modern recurrent neural networks (RNNs)** or **long short-term memory (LSTM)** networks build on ideas from Hopfield networks but introduce mechanisms (like gating) to improve memory retention and capacity for more complex tasks.



Thank You